Gestión de Procesos y Planificación de CPU, ampliado con Gestión de Memoria

Carlos Andrés Ibañez Barrios
Shaadon Jiménez Flores

Docente
Mag. Sergio Luis Lubo Argumedo

Universidad Cooperativa de Colombia
Facultad de Ingenierías
Pregrado de Ingeniería de Software
Santa Marta, Colombia
2026

1. Descripción del Problema

El tema seleccionado corresponde a la Opción 1 del proyecto final: Gestión de Procesos y Planificación de CPU, ampliada a solicitud del docente para incluir Gestión de Memoria con paginación.

En los sistemas computacionales multiprogramados, el procesador (CPU) y la memoria principal (RAM) son los recursos físicos más restringidos y demandados por los programas en ejecución. Cuando múltiples procesos compiten por el tiempo del procesador, la ausencia de una política disciplinada de planificación provoca patologías graves en el sistema. Entre ellas destaca el denominado efecto convoy, donde un proceso con una ráfaga de cómputo extensa monopoliza el procesador y condena a múltiples procesos breves a esperas inaceptables, incrementando los tiempos de retorno y degradando la interactividad percibida por los usuarios (Silberschatz et al., 2018).

De manera simultánea, la asignación tradicional contigua en memoria principal genera fragmentación externa, una condición donde la memoria libre total es teóricamente suficiente para albergar un proceso, pero se encuentra dispersa en múltiples particiones pequeñas y no contiguas que impiden su asignación (Tanenbaum & Bos, 2015). Para subsanar esta debilidad, los sistemas operativos modernos recurren a la memoria virtual paginada. Sin embargo, este mecanismo traslada el problema hacia la gestión de tablas de páginas, la minimización de la fragmentación interna en los marcos de memoria y la selección de políticas óptimas de reemplazo ante fallos de página. El presente proyecto aborda la construcción y evaluación de un simulador ciclo a ciclo en Python 3 que integra ambos subsistemas para analizar cuantitativamente su desempeño.

2. Justificación y Fundamentación Teórica

La planificación de la CPU se fundamenta en la naturaleza empírica del ciclo de ráfagas CPU-E/S, el cual establece que la ejecución de cualquier proceso consiste en una alternancia periódica entre cómputo en procesador y espera por operaciones de transferencia en periféricos (Silberschatz et al., 2018). Un planificador a corto plazo eficiente debe aprovechar los intervalos en que un proceso se bloquea por E/S para despachar a otro proceso listo, elevando el coeficiente de utilización del hardware y garantizando una asignación equitativa del procesador.

Por su parte, la paginación pura divide el espacio de direcciones virtuales del proceso en bloques de tamaño fijo denominados páginas, mientras que la memoria física se particiona en bloques del mismo tamaño denominados marcos. La Unidad de Manejo de Memoria (MMU) efectúa la traducción dinámica de direcciones mediante una tabla de páginas, eliminando por diseño la fragmentación externa (Stallings, 2018). No obstante, cuando la memoria física es inferior a la demanda agregada de los procesos, la presencia de fallos de página exige algoritmos de reemplazo basados en el principio de localidad de referencia. El algoritmo Least Recently Used (LRU) asume que las páginas referenciadas recientemente volverán a ser utilizadas a corto plazo, aventajando al esquema elemental First-In, First-Out (FIFO), el cual desaloja páginas basándose únicamente en su orden cronológico de ingreso a la memoria física.

3. Objetivos del Proyecto

3.1. Objetivo General

Diseñar, implementar y evaluar un simulador determinista de tiempo discreto ciclo a ciclo (reloj tick-by-tick) en Python 3 que integre la planificación de procesos en CPU (FCFS, SJF no expropiativo y Round Robin) y la gestión de memoria virtual con paginación pura (traducción MMU en 4 pasos, cálculo de fragmentación y reemplazo FIFO frente a LRU), analizando cuantitativamente el rendimiento, la sobrecarga por conmutación y la equidad distributiva sobre tres escenarios reproducibles.

3.2. Objetivos Específicos

1. Modelar el ciclo de vida del proceso mediante un Bloque de Control de Proceso (PCB) explícito estructurado bajo cinco estados formales: NUEVO, LISTO, EJECUCION, BLOQUEADO y FINALIZADO, con trazabilidad completa de eventos en logs auditables.

2. Implementar el costo temporal de cambio de contexto como parámetro configurable (costo_cs = 1 tick y costo_cs = 0), asegurando consistencia matemática rigurosa entre el tiempo de espera y el tiempo de respuesta en entornos no expropiativos.

3. Desarrollar la Unidad de Manejo de Memoria (MMU) simulada con el procedimiento estándar de 4 pasos para traducir direcciones virtuales a físicas y detectar fallos de página y violaciones de segmento (Segmentation Fault).

4. Evaluar empíricamente la efectividad de las políticas de reemplazo FIFO y LRU, comprobando la ventaja de LRU ante patrones con localidad temporal y analizando el fenómeno de homogeneización derivado de la rotación temporal en Round Robin.

4. Diseño General de la Solución

La solución se concibe bajo una arquitectura modular de tiempo discreto en Python 3, sin dependencias externas más allá de matplotlib y pandas para la presentación tabular y gráfica. Cada avance unitario del reloj entero del simulador (t = 0, 1, 2, ...) gobierna sincrónicamente la admisión de nuevos procesos, la ejecución en procesador, la traducción de referencias de memoria y la resolución de transferencias de E/S.

El sistema se organiza en seis módulos cohesivos: pcb.py (administra los atributos del proceso y su máquina de estados), planificadores.py (implementa la clase base intercambiable y los algoritmos FCFS, SJF y Round Robin), memoria.py (contiene la estructura de marcos físicos, tablas de páginas por proceso, MMU y estrategias FIFO y LRU), simulador.py (orquesta las colas y el reloj discreto), metricas.py (computa TAT, WT, RT, equidad y fragmentación) y visualizador.py (construye los diagramas de Gantt).

Durante la fase de cómputo en CPU, la MMU ejecuta en cada ciclo el procedimiento canónico de 4 pasos para resolver la dirección física correspondiente, validando estrictamente que el número de página no exceda los límites asignados al proceso:

```
# Procedimiento canonico de traduccion en 4 pasos (MMU)
num_pagina = dir_virtual // self.tamano_pagina
desplazamiento = dir_virtual % self.tamano_pagina
dir_fisica = (id_marco * self.tamano_pagina) + desplazamiento
```

Si el número de página calculado excede el tamaño de la tabla del proceso, el sistema detona inmediatamente una excepción de violación de segmento (Segmentation Fault / IndexError) y registra el evento en el log de la MMU, emulando con precisión el comportamiento de las trampas de hardware en arquitecturas reales.

5. Algoritmos y Estrategias Comparadas

En el subsistema de procesador se seleccionaron tres estrategias clásicas con distintas propiedades de equidad y sobrecarga. En primer término, First-Come, First-Served (FCFS) opera como una cola FIFO no expropiativa simple, atendiendo a los procesos en estricto orden cronológico de arribo. En segundo lugar, Shortest Job First (SJF) en su variante no expropiativa selecciona en cada despacho al proceso cuya ráfaga de CPU inmediata sea la más breve, aplicando como reglas deterministas de desempate el menor tiempo de llegada y el menor identificador numérico (PID). Finalmente, Round Robin (RR) asigna la CPU en rodajas de tiempo discretas denominadas quantum (q). Si el quantum concluye antes de terminar la ráfaga, el proceso es expropiado preventivamente. Ante la concurrencia simultánea entre el vencimiento de quantum de un proceso activo y el arribo de un nuevo proceso a la cola de listos, la política adoptada posiciona primero al proceso entrante, optimizando su tiempo de respuesta inicial.

En el subsistema de memoria se confrontan dos algoritmos de reemplazo de páginas con un tamaño de marco fijado en 4 KB (4096 bytes). El algoritmo FIFO mantiene un registro del instante de carga de cada marco y desaloja la página de mayor antigüedad temporal. En contraste, el algoritmo LRU actualiza la estampa de tiempo en cada lectura o escritura a la dirección física, seleccionando como víctima a la página cuyo acceso previo sea el más remoto en la línea de tiempo del reloj simulado.

6. Escenarios y Métricas de Prueba

La evaluación cuantitativa se estructura en tres escenarios experimentales deterministas almacenados en la suite de pruebas. El Escenario 1 (Carga Ligera con E/S) evalúa cuatro procesos donde dos de ellos (P1 y P3) realizan operaciones de entrada y salida intermedias, forzando la transición al estado BLOQUEADO en una memoria holgada de 8 marcos (32 KB). El Escenario 2 (Carga Mixta y Efecto Convoy) introduce un proceso monolítico P1 con ráfaga de 20 ticks en t = 0 seguido por cuatro procesos cortos de 1 a 3 ticks; simultáneamente en memoria, P1 ejecuta el patrón de referencias de Silberschatz en una RAM limitada a 4 marcos (16 KB) para provocar colisiones de reemplazo. El Escenario 3 (Alta Concurrencia) involucra ocho procesos heterogéneos y evalúa la sensibilidad de Round Robin con quantum q = 2, 4 y 8 ticks, contrastando corridas con sobrecarga real (costo_cs = 1) frente a costo ideal (costo_cs = 0).

Las métricas computadas en cada corrida comprenden: el Tiempo de Retorno (Turnaround Time, TAT = T_fin - T_llegada), el Tiempo de Espera (Waiting Time, WT = Ticks en estado LISTO), el Tiempo de Respuesta (Response Time, RT = T_inicio - T_llegada), la Desviación Estándar de WT (métrica formal de equidad distributiva) y el indicador de inanición estadística (alertas emitidas si la razón WT/CPU supera o iguala 4.0x). En memoria se calcula la Tasa de Aciertos (Hit Rate, HR = Aciertos / Accesos * 100%), la Tasa de Fallos (Page Fault Rate, PFR = 100% - HR) y la Fragmentación Interna acumulada (memoria asignada menos memoria requerida).

7. Repositorio o Carpeta de Trabajo

El código fuente completo, los scripts de automatización, las trazas de auditoría y los documentos técnicos que integran la solución se encuentran organizados bajo control de versiones. El acceso al repositorio del proyecto se encuentra disponible a través del siguiente enlace oficial: **[Link del repositorio: por agregar]**.

La estructura interna del directorio de trabajo sigue una arquitectura modular y reproducible, diseñada para desacoplar la lógica de simulación, la gestión de memoria, las herramientas de medición y los artefactos de salida generados:

```
proyecto_so_cpu/
|-- pcb.py                     # Modelo del proceso y maquina de estados (5 estados)
|-- planificadores.py          # Politicas de CPU: FCFS, SJF y Round Robin
|-- memoria.py                 # Paginacion, MMU (4 pasos) y reemplazo FIFO / LRU
|-- simulador.py               # Motor determinista de simulacion ciclo a ciclo
|-- metricas.py                # Calculo de TAT, WT, RT, equidad y fragmentacion
|-- visualizador.py            # Generacion de graficos y diagramas de Gantt
|-- main.py                    # Orquestador general y suite de pruebas de escenarios
|-- generar_pdf.py             # Generador del informe tecnico formal
|-- generar_revision_preliminar.py # Generador del presente documento de revision APA 7
|-- informe_tecnico.pdf        # Documento del informe tecnico consolidado en PDF
|-- README.md                  # Documentacion de arquitectura, instalacion y ejecucion
|-- .gitignore                 # Exclusion de cache (__pycache__/) y artefactos locales
`-- resultados/
    |-- logs/                  # Trazas de auditoria ciclo a ciclo por escenario (.txt)
    |-- metricas/              # Tablas de resultados consolidadas en formato CSV
    `-- graficos/              # Diagramas de Gantt y graficas comparativas (.png)
```

Esta organización garantiza que cualquier ejecución a través de main.py regenere de forma limpia y determinista la totalidad de los datos analizados, permitiendo verificar de manera inmediata la consistencia entre los logs de eventos, las tablas métricas consolidadas y las representaciones visuales.

8. Plan de Trabajo y Cronograma

El desarrollo del proyecto se organizó en cinco fases secuenciales estructuradas para garantizar entregables verificables y mitigar riesgos técnicos. La Fase 1 comprendió la auditoría conceptual y diseño de la máquina de estados y PCB. La Fase 2 abarcó la implementación y verificación del motor de planificación de CPU, logrando la consistencia exacta WT == RT en entornos no expropiativos. La Fase 3 se centró en la arquitectura de memoria virtual con paginación pura y la implementación de las clases de reemplazo FIFO y LRU. La Fase 4 contempló la auditoría de no-regresión y el blindaje contra excepciones (validaciones de quantum positivo, ráfagas no nulas y detección de Segmentation Fault). Finalmente, la Fase 5 comprende la redacción del informe técnico final, compilación de gráficos y preparación de la sustentación oral.

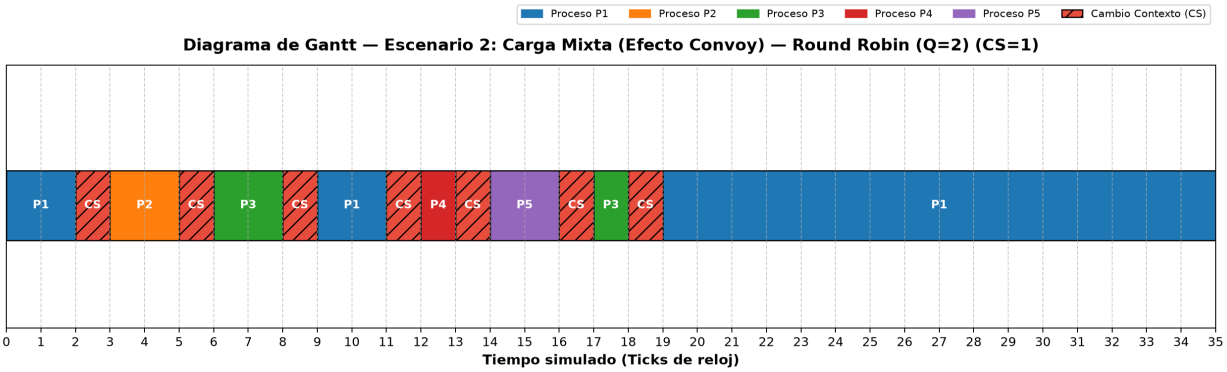
9. Avance Funcional Inicial o Prueba de Concepto

A la fecha de la presente entrega, el simulador se encuentra totalmente implementado y operativo a través del orquestador main.py. El sistema ejecuta con éxito la totalidad de las pruebas de CPU y memoria, generando automáticamente los registros de transiciones en texto, los archivos tabulares consolidados en formato CSV y las representaciones visuales correspondientes.

Como evidencia empírica directa de funcionamiento, la Figura 1 ilustra el Diagrama de Gantt generado por el simulador para el Escenario 2 bajo la política Round Robin con quantum q = 2 ticks y sobrecarga de cambio de contexto costo_cs = 1 tick. En el diagrama se aprecia con claridad el desalojo preventivo del proceso monolítico P1, los intervalos intercalados de sobrecarga (etiquetados como CS) y la rápida atención de los procesos interactivos breves (P2 a P5), logrando una reducción del 73.1% en el tiempo de respuesta frente al algoritmo FCFS.

Figura 1

Diagrama de Gantt del Escenario 2 bajo Planificación Round Robin (q = 2, costo_cs = 1).



Nota. Registro cronológico de ejecución generado automáticamente por visualizador.py en resultados/graficos/. Evidencia la alternancia equitativa entre procesos y los ticks de penalización por cambio de contexto.

Asimismo, las pruebas de aserción automatizadas incorporadas al final de la ejecución de main.py certificaron que en los algoritmos no expropiativos sin E/S el tiempo de espera promedio coincide exactamente con el tiempo de respuesta promedio (FCFS: WT = 18.60, RT = 18.60; SJF: WT = 17.80, RT = 17.80), y que en el módulo de memoria LRU superó a FIFO alcanzando una tasa de aciertos del 46.43% frente al 39.29% en el Escenario 2, confirmando la solidez de la prueba de concepto desarrollada.

Referencias

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). John Wiley & Sons.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson.

Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.